ΠΕΡΙΣΣΟΤΕΡΑ ΓΙΑ ΤΙΣ ΜΕΘΟΔΟΥΣ

ΣΤΗΝ C++

ΜΕΘΟΔΟΙ ΣΤΙΓΜΙΟΤΥΠΩΝ

(στην «αργκό» της C++)

ή

ΡΟΥΤΙΝΕΣ ΠΟΥ ΧΕΙΡΙΖΟΝΤΑΙ ΜΙΑ ΕΓΓΡΑΦΗ ΑΠΟ ΜΙΑ ΚΛΑΣΗ

(στην «αργκό» της C++)

ή απλά

ΡΟΥΤΙΝΕΣ ΠΟΥ ΧΕΙΡΙΖΟΝΤΑΙ ΜΙΑ ΕΓΓΡΑΦΗ ΣΕ ΕΝΑ ΑΡΧΕΙΟ

Οι μέθοδοι οι οποίες επικοινωνούν με τα στοιχεία της κλάσης ονομάζονται **μέθοδοι στιγμιοτύπων (instance methods)** ή **μη-στατικές μέθοδοι (non-static methods)**.

Μία μέθοδος η οποία δεν έχει την λέξη static ονομάζεται μέθοδος στιγμιοτύπου (instance method).

Για να κληθεί μία μέθοδος στιγμιοτύπου, θα πρέπει να χρησιμοποιηθεί ένα αντικείμενο (στιγμιότυπο) της κλάσης στην οποία είναι ορισμένη.

Ο κώδικας των μεθόδων στιγμιοτύπων αλληλεπιδρά με τα πεδία του κάθε αντικειμένου από το οποίο αυτή καλείται.

```
System.out.println(c1.getSpeed());
c2.setSpeed(100);
c3.print();
```

μέσω της αναφοράς c2, που αφορά το αντικείμενο(στιγμιότυπο) BK-457, καλείται η μέθοδος-ρουτίνα SetSpeed και του κάνει την ταχύτητα 100

```
Car c2 = new Car("BK3457", "Audi", 125);
```

```
// Οι μέθοδοι τύπου getter και setter αποτελούν εξ'ορισμού μεθόδους στιγμιοτύπων
// αφού αναφέρονται (επιστρέφουν ή ενημερώνουν) στις τιμές των πεδίων κάθε φορά
//  του  αντικειμένου από το οποίο καλούνται.
```

Η ΕΝΝΟΙΑ STATIC ΣΤΑ ΠΕΔΙΑ

- Ένα πεδίο σε μία κλάση μπορεί να δηλωθεί ως static (στατικό) ως εξής:

τροποποιητής static τύπος_πεδίου όνομα_πεδίου;

(public, private, protected, package)

```
public  int hp;
public  String brand;
```

```
public static int hp;
public static String brand;
```

- Η έννοια του static πεδίου είναι η εξής: το πεδίο ανήκει συνολικά σε όλη την κλάση και όχι σε αντικείμενο ξεχωριστά. Αυτό πρακτικά σημαίνει ότι κάθε φορά που το πεδίο αλλάζει τιμή, όλα τα αντικείμενα της κλάσης έχουν αυτή την τιμή στο εν λόγω πεδίο.

- Για να αναφερθούμε σε ένα static πεδίο, δεν είναι ανάγκη να δημιουργήσουμε αντικείμενο της εν λόγω κλάσης. Αν το πεδίο έχει τροποποιητή public, μπορούμε να αναφερθούμε σε αυτό από την κλάση κατευθείαν.

- Συνήθως ένα πεδίο μίας κλάσης το οποίο είναι final (σταθερά) δηλώνεται επίσης ως static και με τον τροποποιητή public, έτσι ώστε να μπορεί να αναφερθεί από την κλάση κατευθείαν.

```
package mathima8;

public class Car
{
  public static int speed;

  public Car(int speed)          // Constructor
    {this.speed = speed;}

  public int getSpeed()          // Getter
    { return speed; }

  public void setSpeed(int speed)      // Sette
    { this.speed = speed; }
}

public class Mathima8
{
  public static void main(String[] args)
  {
    Car c1 = new Car(100);
    Car c2 = new Car(70);

    System.out.println(c1.getSpeed()); // Θα τυπώσει 100
    System.out.println(c2.getSpeed()); // Θα τυπώσει 70

    Car.speed = 120; // το πεδίο speed παίρνει την τιμή 120
    System.out.println(c1.getSpeed()); // Θα τυπώσει 120
    System.out.println(c2.getSpeed()); // Θα τυπώσει 120

    System.out.println(Car.speed); // Θα τυπώσει 120
                                   αναφορά στο πεδίο
                                   κατευθείαν από την κλάση
  }
}
```

δεν αναφέρομαι δηλαδή, σε μια αναφορά τύπου c1, (δηλαδή σε κάποιο συγκεκριμένο αυτοκίνητο)

αλλά αναφέρομαι σε όλη την κλάση (βάζοντας μπροστά το Car)

και κατά συνέπεια όλα τα αυτοκίνητα (c1, c2, ...) αποκτούν την ίδια ταχύτητα

Η ΕΝΝΟΙΑ STATIC ΣΤΙΣ ΜΕΘΟΔΟΥΣ-ΡΟΥΤΙΝΕΣ

- Μία μέθοδος μπορεί να δηλωθεί ως static (στατική), εισάγοντας την λέξη static ακριβώς πριν από τον τον τύπο της στην υπογραφή της.

- Μία static μέθοδος ονομάζεται μέθοδος κλάσης γιατί μπορεί να καλεσθεί από την κλάση στην οποία είναι δηλωμένη, χωρίς να είναι αναγκαία η δημιουργία αντικειμένου της κλάσης:

όνομα_κλάσης.όνομα_μεθόδου(τυχόν ορίσματα);

εάν βάλουμε δηλαδή το STATIC μπροστά στην μέθοδο, μπορούμε στη συνέχεια, όταν την καλούμε να μην αναφερόμαστε (κατά την κλήση της) σε κάποια συγκεκριμένη αναφορά (π.χ. c1) προς ένα αντικείμενο (π.χ. το αυτοκίνητο ΥΗΑ-3456), αλλά απλά να την εκτελούμε και να μας γυρίζει κάτι που υπολόγισε

- Δηλώνουμε μία μέθοδο ως static, αν θέλουμε αυτή να εκτελέσει κάποιον γενικό αλγόριθμο ο οποίος δεν εκτελείται στα πεδία των αντικειμένων μιας κλάσης (π.χ. υπολογισμός τετραγωνικής ρίζας, υπολογισμός απόλυτης τιμής ενός αριθμού, κτλ.).

- **Μία static μέθοδος μπορεί να εκτελέσει ενέργειες μόνο στα static πεδία της κλάσης της.**

```
package mathima8;
public class MathClass
{    // Υπολογίζει την y-οστή δύναμη του αριθμού x (x,y ακέραιοι)
     public static double pow(int x, int y)
     {   int n = y;
         if (y < 0)  n = -y;
         double res = 1;
         for (int i = 1; i <= n; i++)  res *= x;
         if (y < 0) res = 1.0/res;
         return res;
     }
}

package mathima8;
public class Mathima8
{
     public static void main(String[] args)
     {
       double p = MathClass.pow(5, 2);   // Κλήση static μεθόδου pow
       System.out.println(p);               κατευθείαν
     }                                      στην κλάση MathClass
 }
```

ΜΕΘΟΔΟΙ-ΡΟΥΤΙΝΕΣ ΠΟΥ ΚΑΛΟΥΝ ΑΛΛΕΣ ΜΕΘΟΔΟΥΣ-ΡΟΥΤΙΝΕΣ

- μία μέθοδος μπορεί να καλέσει μία άλλη, ο τρόπος κλήσης διαφέρει ανάλογα με το αν πρόκειται για μέθοδο στιγμιοτύπου ή μέθοδο κλάσης.

- Η κλήση μπορεί να γίνει περισσότερα εύκολα αν πρόκειται για μεθόδους οι οποίες βρίσκονται ορισμένες μέσα στην ίδια κλάση.

- Αν από το σώμα μίας μη-στατικής μεθόδου (μεθόδου στιγμιοτύπου) A επιθυμούμε να καλέσουμε μία μέθοδο B η οποία βρίσκεται ορισμένη στην ίδια κλάση, την καλούμε χωρίς να δημιουργήσουμε αντικείμενο της κλάσης, γράφοντας απλά το όνομά της,

  όνομα_μεθόδου(τυχόν ορίσματα);

- Αν από το σώμα μίας στατικής μεθόδου (μέθοδος κλάσης) A επιθυμούμε να καλέσουμε μία μέθοδο B η οποία βρίσκεται ορισμένη στην ίδια κλάση, την καλούμε με τον ίδιο ακριβώς τρόπο όπως και παραπάνω. Ο μόνος περιορισμός είναι οτι και η μέθοδος B θα πρέπει να είναι στατική.

- **Μία στατική μέθοδος δεν μπορεί να καλέσει εσωτερικά μεθόδους της ίδιας κλάσης οι οποίες δεν είναι στατικές.**

```
package mathima8;
public class Car
{
    private int speed;
    public Car(int speed)
     {    this.speed = speed;
     }
    public int getSpeed()
     { return speed; }
    public void speedUp()
     { speed += 10; }
    public void start()
    {
     if (speed == 0)
        speedUp();
    }
}
package mathima8;
public class Mathima8
{
  public static void main(String[] args)
  {
    Car c = new Car(0);
    c.start();
    System.out.println(c.getSpeed());
  }
}
```

ΚΛΗΣΗ ΜΗ ΣΤΑΤΙΚΩΝ ΜΕΘΟΔΩΝ μεταξυ τους

```
package mathima8;
public class Mathima8
{
  public static int abs(int x)
  {
    if (x < 0)
        return -1*x;
    else
        return x;
  }

  public int pow2(int x)
  {    return x*x;
  }

  public static void main(String[] args)
  {
   int k = abs(-5);          // κλήση της στατικής μεθόδου abs που
   System.out.println(k);    // βρίσκεται ορισμένη στην ίδια κλάση

   Mathima8 obj = new Mathima8(); //για την κλήση μη στατικών
   int y = obj.pow2(5);           // μεθόδων είναι απαραίτητη
   System.out.println(y);         // η δημιουργία νέου αντικειμένων
                                  // ακόμα κι αν αυτές βρίσκονται
   int z = pow2(10);  // η pow2    // στην ίδια κλάση
  }        είναι λάθος διότι  δεν είναι στατική
}
```

ΣΥΝΕΧΟΜΕΝΕΣ ΚΛΗΣΕΙΣ ΜΕΘΟΔΩΝ-ΡΟΥΤΙΝΩΝ

- Αν ο επιστρεφόμενος τύπος μιας μεθόδου είναι μια αναφορά, μπορούμε να καλέσουμε μεθόδους από την κλάση της εν λόγω αναφοράς κατευθείαν, χρησιμοποιώντας τον τελεστή τελεία

  αναφορά.κλήση_μεθόδου_1().κλήση_μεθόδου_2().κλήση_μεθόδου_3()...

```
package mathima8;
public class Driver
{
  private String name;
  public Driver(String name)
  {
    this.name = name;
  }
  public String getName()
  { return name; }
}
```

```
package mathima8;
public class Car
{
  private String plateNumber;
  private Driver driver;
  public Car(String plateNumber, Driver d)
  {   this.plateNumber = plateNumber;
      this.driver = d;
  }
  public String getPlateNumber()
  { return plateNumber; }
  public Driver getD()
  { return driver; }
}
```

```
package mathima8;
public class Mathima8
{
  public static void main(String[] args)
  {
    Driver d = new Driver("Nik");
    Car c = new Car("AE1475",d);
    System.out.println(c.getD().getName());
  }
}
```